1.Build a fully connected neural network (CNN) and a convolutional neural network (CNN) for classifying 10 classes of images.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

# Load and preprocess CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize the image data to range [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Define the Fully Connected Neural Network (CNN)
def build_fcnn():
    model = Sequential([
        Flatten(input_shape=(32, 32, 3)),  # Flatten the input images
        Dense(512, activation='relu'),
        Dropout(0.5),  # Regularization to prevent overfitting
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')  # Output layer for 10 classes
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Define the Convolutional Neural Network (CNN)
def build_cnn():
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
```

```
        Dropout(0.25),

        Conv2D(128, (3, 3), activation='relu'),
        Flatten(),  # Flatten the output before feeding into Dense layers

        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')  # Output layer for 10 classes
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Instantiate and train the Fully Connected Neural Network
print("Training Fully Connected Neural Network (FCNN)...")
fcnn_model = build_fcnn()
fcnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
fcnn_loss, fcnn_accuracy = fcnn_model.evaluate(x_test, y_test, verbose=0)
print(f"FCNN Test Accuracy: {fcnn_accuracy * 100:.2f}%")

# Instantiate and train the Convolutional Neural Network
print("Training Convolutional Neural Network (CNN)...")
cnn_model = build_cnn()
cnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
cnn_loss, cnn_accuracy = cnn_model.evaluate(x_test, y_test, verbose=0)
print(f"CNN Test Accuracy: {cnn_accuracy * 100:.2f}%")
```

Answer:

Training Fully Connected Neural Network (FCNN)...

To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 15s 21ms/step - accuracy: 0.1590 - loss: 2.2975 - val_accuracy: 0.2596 - val_loss: 1.9755
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.1885 - loss: 2.1036 - val_accuracy: 0.2684 - val_loss: 1.9816
Epoch 3/10

625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2026 - loss: 2.0775 - val_accuracy: 0.2598 - val_loss: 2.0033
Epoch 4/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2098 - loss: 2.0607 - val_accuracy: 0.2776 - val_loss: 1.9760
Epoch 5/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2229 - loss: 2.0393 - val_accuracy: 0.2739 - val_loss: 1.9938
Epoch 6/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2346 - loss: 2.0203 - val_accuracy: 0.2580 - val_loss: 1.9869
Epoch 7/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2361 - loss: 2.0021 - val_accuracy: 0.3063 - val_loss: 1.9566
Epoch 8/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2408 - loss: 2.0071 - val_accuracy: 0.2901 - val_loss: 1.9660
Epoch 9/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2386 - loss: 1.9995 - val_accuracy: 0.2938 - val_loss: 1.9889
Epoch 10/10
625/625 ——————————————————— 13s 21ms/step - accuracy: 0.2498 - loss: 1.9981 - val_accuracy: 0.2909 - val_loss: 1.9824
FCNN Test Accuracy: 29.47%

Training Convolutional Neural Network (CNN)...
Epoch 1/10
625/625 ——————————————————— 27s 40ms/step - accuracy: 0.2789 - loss: 1.9216 - val_accuracy: 0.5115 - val_loss: 1.3595
Epoch 2/10
625/625 ——————————————————— 27s 43ms/step - accuracy: 0.5084 - loss: 1.3659 - val_accuracy: 0.5862 - val_loss: 1.1784
Epoch 3/10
625/625 ——————————————————— 29s 47ms/step - accuracy: 0.5743 - loss: 1.1989 - val_accuracy: 0.6203 - val_loss: 1.0726
Epoch 4/10
625/625 ——————————————————— 30s 47ms/step - accuracy: 0.6158 - loss: 1.0885 - val_accuracy: 0.6562 - val_loss: 0.9784
Epoch 5/10
625/625 ——————————————————— 30s 49ms/step - accuracy: 0.6419 - loss: 1.0159 - val_accuracy: 0.6757 - val_loss: 0.9229
Epoch 6/10
625/625 ——————————————————— 30s 47ms/step - accuracy: 0.6661 - loss: 0.9430 - val_accuracy: 0.6856 - val_loss: 0.9155

```
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31s 50ms/step - accuracy: 0.6921 - loss:
0.8829 - val_accuracy: 0.7063 - val_loss: 0.8505
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 29s 47ms/step - accuracy: 0.7042 - loss:
0.8420 - val_accuracy: 0.7159 - val_loss: 0.7990
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 29s 47ms/step - accuracy: 0.7194 - loss:
0.8012 - val_accuracy: 0.7165 - val_loss: 0.8102
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 30s 48ms/step - accuracy: 0.7236 - loss:
0.7849 - val_accuracy: 0.7299 - val_loss: 0.7941

CNN Test Accuracy: 71.32%
```

2.Train and test your FCNN and CNN by the Fashion dataset. Discuss your results by comparing performance between two types of networks.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical

# Load and preprocess Fashion MNIST dataset
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Reshape and normalize the image data
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Fully Connected Neural Network (FCNN)
def build_fcnn():
    model = Sequential([
        Flatten(input_shape=(28, 28, 1)),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
```

```
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Convolutional Neural Network (CNN)
def build_cnn():
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        Dropout(0.25),

        Flatten(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Instantiate models
fcnn_model = build_fcnn()
cnn_model = build_cnn()

# Train and evaluate FCNN
print("Training FCNN...")
fcnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
fcnn_loss, fcnn_accuracy = fcnn_model.evaluate(x_test, y_test, verbose=0)
print(f"FCNN Test Accuracy: {fcnn_accuracy * 100:.2f}%")

# Train and evaluate CNN
print("Training CNN...")
cnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
cnn_loss, cnn_accuracy = cnn_model.evaluate(x_test, y_test, verbose=0)
print(f"CNN Test Accuracy: {cnn_accuracy * 100:.2f}%")
```

Answer:

Training FCNN…

Epoch 1/10
750/750 ——————————————————— 8s 9ms/step - accuracy: 0.6823 - loss: 0.8803 - val_accuracy: 0.8376 - val_loss: 0.4426
Epoch 2/10
750/750 ——————————————————— 6s 8ms/step - accuracy: 0.8216 - loss: 0.4944 - val_accuracy: 0.8542 - val_loss: 0.4003
Epoch 3/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8340 - loss: 0.4574 - val_accuracy: 0.8603 - val_loss: 0.3769
Epoch 4/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8456 - loss: 0.4241 - val_accuracy: 0.8627 - val_loss: 0.3675
Epoch 5/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8483 - loss: 0.4128 - val_accuracy: 0.8684 - val_loss: 0.3640
Epoch 6/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8541 - loss: 0.3978 - val_accuracy: 0.8715 - val_loss: 0.3536
Epoch 7/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8552 - loss: 0.3961 - val_accuracy: 0.8704 - val_loss: 0.3612
Epoch 8/10
750/750 ——————————————————— 7s 9ms/step - accuracy: 0.8637 - loss: 0.3766 - val_accuracy: 0.8647 - val_loss: 0.3585
Epoch 9/10
750/750 ——————————————————— 8s 10ms/step - accuracy: 0.8619 - loss: 0.3767 - val_accuracy: 0.8739 - val_loss: 0.3472
Epoch 10/10
750/750 ——————————————————— 10s 10ms/step - accuracy: 0.8677 - loss: 0.3598 - val_accuracy: 0.8796 - val_loss: 0.3385

FCNN Test Accuracy: 86.97%

Training CNN...
Epoch 1/10
750/750 ——————————————————— 22s 27ms/step - accuracy: 0.6753 - loss: 0.8874 - val_accuracy: 0.8612 - val_loss: 0.3991

```
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 20s 27ms/step - accuracy: 0.8376 - loss:
0.4464 - val_accuracy: 0.8708 - val_loss: 0.3484
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21s 28ms/step - accuracy: 0.8637 - loss:
0.3727 - val_accuracy: 0.8849 - val_loss: 0.3056
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.8734 - loss:
0.3398 - val_accuracy: 0.8937 - val_loss: 0.2845
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26s 34ms/step - accuracy: 0.8799 - loss:
0.3204 - val_accuracy: 0.8966 - val_loss: 0.2753
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 25s 33ms/step - accuracy: 0.8854 - loss:
0.3076 - val_accuracy: 0.9003 - val_loss: 0.2672
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23s 30ms/step - accuracy: 0.8932 - loss:
0.2879 - val_accuracy: 0.9037 - val_loss: 0.2577
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22s 30ms/step - accuracy: 0.8979 - loss:
0.2780 - val_accuracy: 0.9057 - val_loss: 0.2514
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21s 28ms/step - accuracy: 0.9013 - loss:
0.2648 - val_accuracy: 0.9061 - val_loss: 0.2513
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23s 30ms/step - accuracy: 0.9030 - loss:
0.2600 - val_accuracy: 0.9046 - val_loss: 0.2495

CNN Test Accuracy: 90.17%
```

3.Build a CNN having a pre-trained MobileNet as backbone to classify 10 classes.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.applications import MobileNet
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

# Load and preprocess the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```
# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Define the MobileNet backbone
def build_model():
    # Load pre-trained MobileNet without the top layer
    backbone = MobileNet(
        input_shape=(32, 32, 3),
        include_top=False,
        weights='imagenet'
    )
    backbone.trainable = False  # Freeze the pre-trained layers

    # Build the full model
    model = Sequential([
        backbone,
        GlobalAveragePooling2D(),
        Dropout(0.5),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')  # 10 classes
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Instantiate the model
model = build_model()

# Train the model
print("Training the model...")
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Answer:

```
Training the model...
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36s 52ms/step - accuracy: 0.1356 - loss:
2.2840 - val_accuracy: 0.2068 - val_loss: 2.2299
Epoch 2/10
```

```
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31s 50ms/step - accuracy: 0.1550 - loss: 2.2503 - val_accuracy: 0.1864 - val_loss: 2.2263
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31s 50ms/step - accuracy: 0.1584 - loss: 2.2462 - val_accuracy: 0.1718 - val_loss: 2.2390
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31s 50ms/step - accuracy: 0.1631 - loss: 2.2432 - val_accuracy: 0.1690 - val_loss: 2.2425
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 34s 54ms/step - accuracy: 0.1600 - loss: 2.2425 - val_accuracy: 0.1772 - val_loss: 2.2308
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35s 56ms/step - accuracy: 0.1631 - loss: 2.2409 - val_accuracy: 0.1753 - val_loss: 2.2305
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36s 57ms/step - accuracy: 0.1628 - loss: 2.2424 - val_accuracy: 0.1633 - val_loss: 2.2431
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37s 59ms/step - accuracy: 0.1661 - loss: 2.2372 - val_accuracy: 0.1686 - val_loss: 2.2408
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36s 58ms/step - accuracy: 0.1637 - loss: 2.2378 - val_accuracy: 0.1699 - val_loss: 2.2320
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36s 58ms/step - accuracy: 0.1633 - loss: 2.2417 - val_accuracy: 0.1715 - val_loss: 2.2344

Test Accuracy: 17.11%
```

4.Train and test your CNN having a pre-trained MobileNet as backbone to classify images of the CIFAR-10 dataset. Discuss your results by comparing performance between transfer_learning + fine tuning and only transfer learning.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.applications import MobileNet
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

# Load and preprocess the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

```
# Normalize pixel values
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Define the MobileNet backbone model
def build_model(trainable_backbone=False):
    # Load pre-trained MobileNet without the top layer
    backbone = MobileNet(
        input_shape=(32, 32, 3),
        include_top=False,
        weights='imagenet'
    )
    backbone.trainable = trainable_backbone  # Freeze or unfreeze backbone

    # Build the full model
    model = Sequential([
        backbone,
        GlobalAveragePooling2D(),
        Dropout(0.5),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')  # 10 classes
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Step 1: Train with Transfer Learning (Backbone Frozen)
print("Training with Transfer Learning (Backbone Frozen)...")
model_tl = build_model(trainable_backbone=False)
history_tl = model_tl.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
test_loss_tl, test_accuracy_tl = model_tl.evaluate(x_test, y_test, verbose=0)
print(f"Transfer Learning Test Accuracy: {test_accuracy_tl * 100:.2f}%")

# Step 2: Train with Transfer Learning + Fine-Tuning (Backbone Unfrozen)
print("Training with Transfer Learning + Fine-Tuning (Backbone Unfrozen)...")
model_ft = build_model(trainable_backbone=True)
history_ft = model_ft.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
```

```
test_loss_ft, test_accuracy_ft = model_ft.evaluate(x_test, y_test, verbose=0)
print(f"Transfer Learning + Fine-Tuning Test Accuracy: {test_accuracy_ft * 100:.2f}%")
```

Answer:

```
Training with Transfer Learning (Backbone Frozen)...
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 43s 56ms/step - accuracy: 0.1333 - loss: 2.2879 - val_accuracy: 0.1962 - val_loss: 2.2362
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 34s 54ms/step - accuracy: 0.1579 - loss: 2.2539 - val_accuracy: 0.1934 - val_loss: 2.2245
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 36s 57ms/step - accuracy: 0.1640 - loss: 2.2444 - val_accuracy: 0.1749 - val_loss: 2.2279
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 47s 75ms/step - accuracy: 0.1673 - loss: 2.2429 - val_accuracy: 0.1698 - val_loss: 2.2252
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 47s 75ms/step - accuracy: 0.1651 - loss: 2.2414 - val_accuracy: 0.1636 - val_loss: 2.2422
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 44s 70ms/step - accuracy: 0.1623 - loss: 2.2406 - val_accuracy: 0.1740 - val_loss: 2.2346
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 75s 58ms/step - accuracy: 0.1642 - loss: 2.2413 - val_accuracy: 0.1530 - val_loss: 2.2463
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 48s 76ms/step - accuracy: 0.1647 - loss: 2.2396 - val_accuracy: 0.1680 - val_loss: 2.2298
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 47s 74ms/step - accuracy: 0.1658 - loss: 2.2378 - val_accuracy: 0.1624 - val_loss: 2.2391
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.1674 - loss: 2.2359 - val_accuracy: 0.1665 - val_loss: 2.2374
Transfer Learning Test Accuracy: 16.71%
Training with Transfer Learning + Fine-Tuning (Backbone Unfrozen)...
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 186s 273ms/step - accuracy: 0.3940 - loss: 2.1888 - val_accuracy: 0.6764 - val_loss: 0.9627
Epoch 2/10
```

```
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 189s 302ms/step - accuracy: 0.6865 -
loss: 0.9738 - val_accuracy: 0.7361 - val_loss: 0.7946
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 186s 298ms/step - accuracy: 0.7572 -
loss: 0.7809 - val_accuracy: 0.7459 - val_loss: 0.7664
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 182s 292ms/step - accuracy: 0.7816 -
loss: 0.6959 - val_accuracy: 0.7373 - val_loss: 0.7992
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 165s 264ms/step - accuracy: 0.8018 -
loss: 0.6293 - val_accuracy: 0.7470 - val_loss: 0.8757
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166s 266ms/step - accuracy: 0.8104 -
loss: 0.6176 - val_accuracy: 0.7796 - val_loss: 0.6916
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 168s 268ms/step - accuracy: 0.8332 -
loss: 0.5419 - val_accuracy: 0.7922 - val_loss: 0.6281
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166s 266ms/step - accuracy: 0.8434 -
loss: 0.5085 - val_accuracy: 0.7801 - val_loss: 0.6752
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166s 265ms/step - accuracy: 0.8460 -
loss: 0.4946 - val_accuracy: 0.7882 - val_loss: 0.6946
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 168s 268ms/step - accuracy: 0.8662 -
loss: 0.4460 - val_accuracy: 0.7582 - val_loss: 0.8129

Transfer Learning + Fine-Tuning Test Accuracy: 75.69%
```

5.Prepare a neural network with appropriate activation functions, so that it has the same architecture as shown in the following Figure-1.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load and preprocess the dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize the dataset (values between 0 and 1)
x_train = x_train.astype('float32') / 255.0
```

```
x_test = x_test.astype('float32') / 255.0

# Flatten the input images (28x28 -> 784)
input_shape = (28, 28)

# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Build the Fully Connected Neural Network (FCNN)
def build_fcnn():
    model = Sequential([
        Flatten(input_shape=input_shape),              # Flatten input (28x28 to 784)
        Dense(512, activation='relu'),                 # Hidden layer 1 with ReLU
        Dropout(0.3),                                  # Dropout for regularization
        Dense(256, activation='relu'),                 # Hidden layer 2 with ReLU
        Dropout(0.3),                                  # Dropout for regularization
        Dense(128, activation='relu'),                 # Hidden layer 3 with ReLU
        Dense(10, activation='softmax')                # Output layer (10 classes)
    ])
    model.compile(optimizer='adam',                    # Optimizer: Adam
                  loss='categorical_crossentropy',     # Loss function: Categorical Crossentropy
                  metrics=['accuracy'])                # Metric: Accuracy
    return model

# Instantiate the FCNN model
fcnn_model = build_fcnn()

# Train the FCNN
print("Training FCNN...")
fcnn_model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

# Evaluate the FCNN
test_loss, test_accuracy = fcnn_model.evaluate(x_test, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Answer:

Training FCNN...
Epoch 1/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9s 9ms/step - accuracy: 0.8355 - loss: 0.5318 - val_accuracy: 0.9626 - val_loss: 0.1223
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9554 - loss: 0.1409 - val_accuracy: 0.9713 - val_loss: 0.0944
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9678 - loss: 0.1076 - val_accuracy: 0.9728 - val_loss: 0.0941
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9752 - loss: 0.0826 - val_accuracy: 0.9744 - val_loss: 0.0885
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9745 - loss: 0.0789 - val_accuracy: 0.9755 - val_loss: 0.0851
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9801 - loss: 0.0624 - val_accuracy: 0.9779 - val_loss: 0.0809
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9811 - loss: 0.0604 - val_accuracy: 0.9786 - val_loss: 0.0788
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.9839 - loss: 0.0526 - val_accuracy: 0.9763 - val_loss: 0.0852
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9846 - loss: 0.0484 - val_accuracy: 0.9770 - val_loss: 0.0926
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9864 - loss: 0.0434 - val_accuracy: 0.9776 - val_loss: 0.0887
Test Accuracy: 97.83%